UNIVERSIDAD DE SAN BUENAVENTURA
Tecnología en Desarrollo de Software · VI semestre
Programación Avanzada y Aplicada

Documento de requisitos, arquitectura y decisiones técnicas

Problema, historias de usuario, requisitos, flujo y decisiones

Proyecto final: **AsistenteIA**, asistente académico móvil con .NET MAUI y una API REST de IA

Autores: Jeferson Wilderman González Tenjo, María Alejandra Gómez y José Leandro Ocampo
Docente: Mg. Sandra Esperanza Sánchez Barragán
Fecha: 16 de octubre de 2026
Versión: rama `main` (versión final del proyecto)

Introducción

Este documento reúne el análisis y el diseño de AsistenteIA: el problema que resuelve, sus usuarios y restricciones, las historias de usuario con criterios de aceptación, los requisitos funcionales RF01–RF10 con su trazabilidad al código, el flujo de navegación, un resumen de la arquitectura y las decisiones técnicas con su justificación.

Problema

Los estudiantes usan asistentes de IA sin conservar lo que preguntaron, sin distinguir qué respuestas verificaron y, a veces, compartiendo datos que no deberían. **AsistenteIA** ofrece un espacio académico para consultar un modelo de IA con plantillas orientadas al estudio. La app guarda un historial local, permite marcar las respuestas contrastadas con fuentes y recuerda los límites y riesgos de la IA.

Usuarios: estudiantes universitarios. **Restricciones:** la app no puede incluir claves secretas en el código, funciona en Windows o Android y depende de un proveedor externo de IA (Google Gemini o Pollinations, seleccionable en Configuración).

Historias de usuario

ID	Historia	Criterios de aceptación
HU01	Como estudiante quiero escribir una pregunta y recibir una respuesta de IA para resolver dudas.	Con un prompt válido y una clave válida, la respuesta aparece y el estado indica "Respuesta recibida" (P01).
HU02	Como estudiante quiero que la app me avise si mi consulta está vacía o es muy larga para no gastar solicitudes.	Un prompt vacío o con menos de 5 caracteres no se envía y se muestra un mensaje (P02). Si supera 2000 caracteres, se rechaza.
HU03	Como estudiante quiero usar plantillas (explicar, resumir, preguntas de repaso) y editarlas.	Hay plantillas iniciales. Puedo crear, editar y eliminar plantillas, y la plantilla elegida se aplica al prompt.
HU04	Como estudiante quiero ver claramente si la app está consultando, si falló o si no hay internet.	Aparece un indicador de carga. Los errores 401, 402, 429 y 5xx, el tiempo agotado y la falta de conexión muestran mensajes comprensibles (P03, P04, P05).
HU05	Como estudiante quiero cancelar una consulta que tarda mucho.	El botón Cancelar detiene la solicitud sin bloquear la interfaz (P06).
HU06	Como estudiante quiero guardar consultas y respuestas para repasarlas después.	Guardar crea un registro en SQLite que aparece en el historial (P07) y que sigue ahí al reiniciar la app (P09).
HU07	Como estudiante quiero buscar, filtrar, ver en detalle y eliminar conversaciones guardadas.	El historial permite buscar por texto y filtrar por categoría. Eliminar pide confirmación (P08). El detalle muestra prompt, respuesta, fecha, categoría y estado de verificación.
HU08	Como estudiante quiero marcar una respuesta como verificada después de contrastarla con fuentes.	El detalle tiene un botón para marcar o desmarcar, y el cambio se guarda (actualización del CRUD).
HU09	Como estudiante quiero registrar mi clave sin que se vea en pantalla.	La clave se escribe en un campo de contraseña y se muestra enmascarada (••••1234). Se puede eliminar, con confirmación.
HU10	Como estudiante quiero saber para qué sirve la app, quién la hizo, qué pasa con mis datos y qué límites tiene.	La sección Acerca de explica finalidad, autoría, fuentes, privacidad y limitaciones.

Requisitos funcionales y trazabilidad

Código	Requisito	Implementación
RF01	Ingresar una pregunta o instrucción	`Views/ChatPage.xaml` (Editor enlazado a `ChatViewModel.Prompt`)
RF02	Validar contenido y longitud (5–2000)	`AiService.ValidatePrompt`, usado en `ChatViewModel.AskAsync` antes de enviar
RF03	Solicitud POST a la API REST	`AiService.SendChatAsync` → `POST chat/completions`
RF04	Estados de carga, respuesta, error y sin conexión	`ChatViewModel.IsBusy/StatusMessage/IsError`, burbuja "Pensando la respuesta..." (`ChatExchangeViewModel.IsPending`), `AiServiceException`, `IConnectivity`
RF05	Guardar consulta y respuesta en SQLite	Botón Guardar de cada respuesta → `ChatViewModel.SaveAsync(exchange)` → `DatabaseService.SaveAsync`
RF06	Consultar, filtrar, ver detalle y eliminar	`HistoryViewModel`, `DetailViewModel`, `DatabaseService.GetConversationsAsync/DeleteAsync`
RF07	Plantillas académicas editables	`TemplatesViewModel`, `PromptTemplate`, tabla `PromptTemplate` en SQLite
RF08	Registrar o actualizar la clave sin mostrarla	`SettingsPage` (`Entry IsPassword="True"`), `ApiKeyService` (SecureStorage)
RF09	Acerca de: finalidad, autoría, fuentes, privacidad y límites	Sección "Acerca de AsistenteIA" en `SettingsPage.xaml`
RF10	Funciona en Windows o Android	Destinos `net10.0-windows10.0.19041.0` y `net10.0-android`. Pruebas en Windows

Flujo de navegación

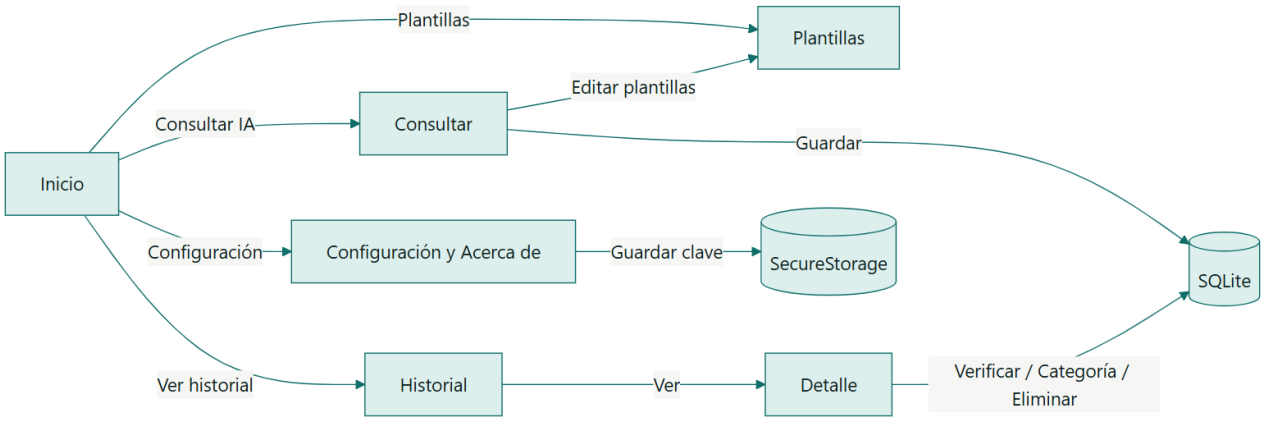


Figura 1. Flujo de navegación entre pantallas y almacenes de datos.

Las cuatro pantallas principales están en la barra de pestañas de Shell. **Detalle** (`detail?id=`) y **Plantillas** (`templates`) son rutas registradas en `AppShell.xaml.cs`.

Pantallas implementadas

Los wireframes iniciales de baja fidelidad evolucionaron a la interfaz final. Estas son las pantallas reales de la versión actual:

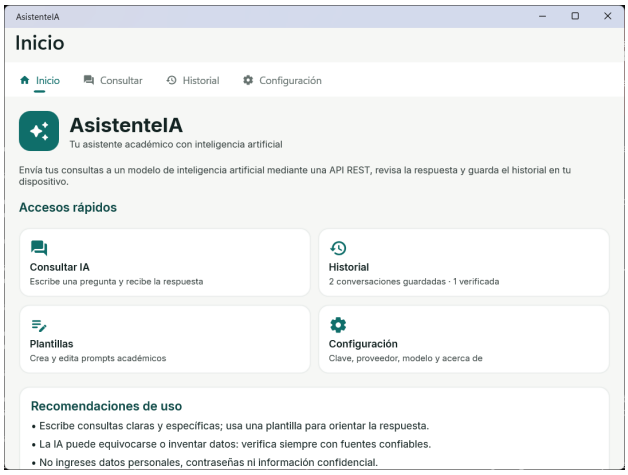


Figura 2. Inicio.

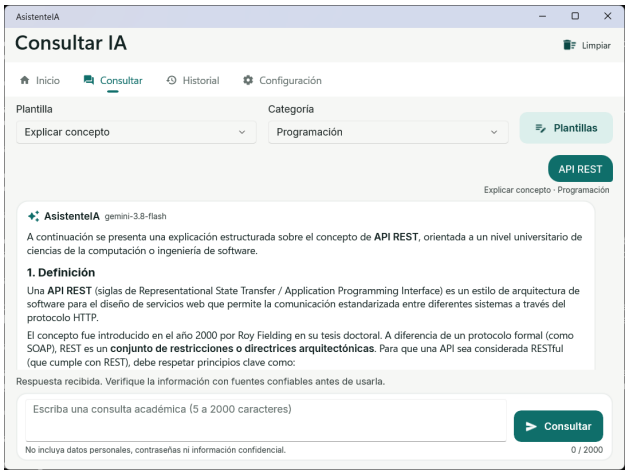


Figura 3. Consultar IA.

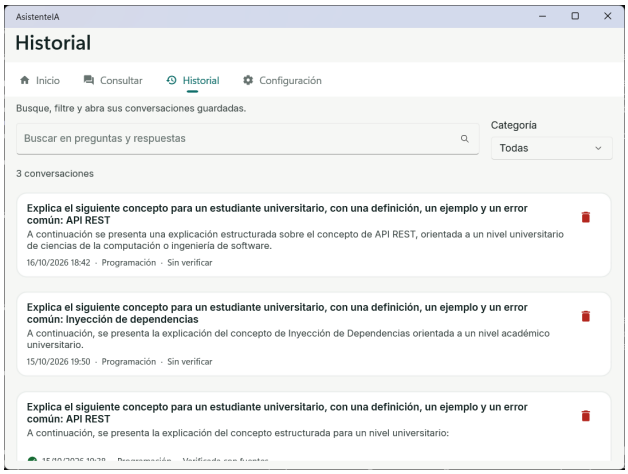


Figura 4. Historial.

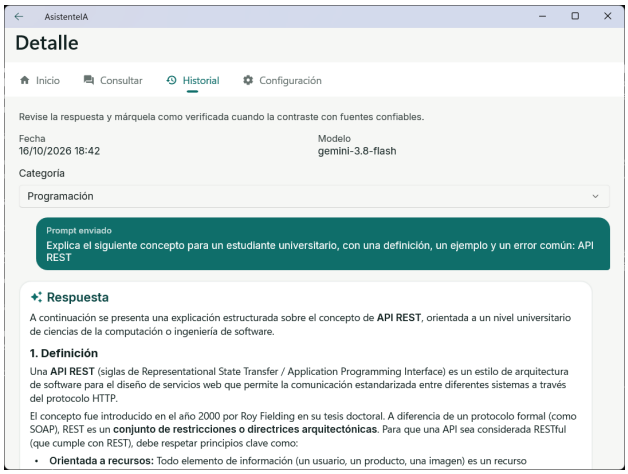


Figura 5. Detalle.

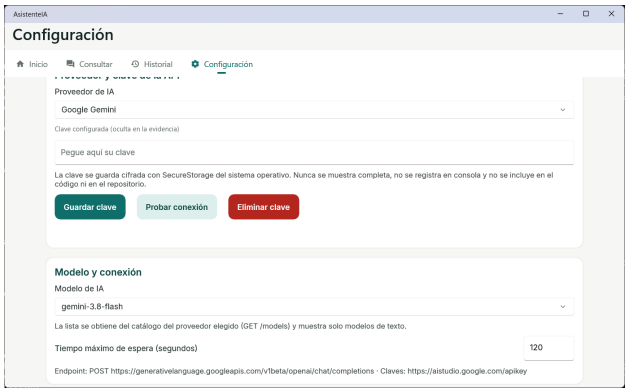


Figura 6. Configuración y Acerca de.

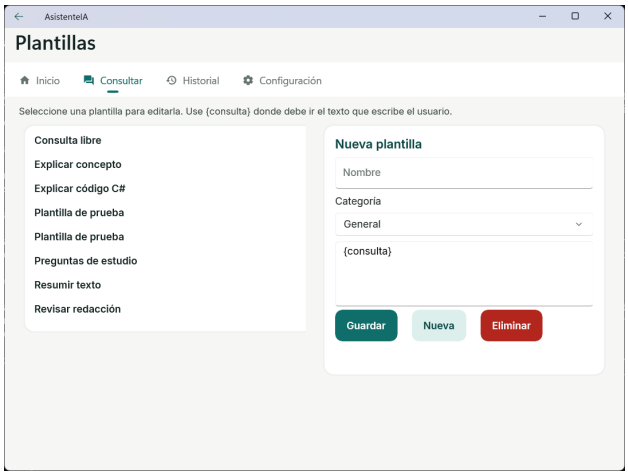


Figura 7. Plantillas.

Arquitectura (resumen)

La solución sigue el patrón MVVM con inyección de dependencias y se organiza en cinco componentes: presentación, servicios y consumo REST, lógica de negocio y modelos, persistencia, y seguridad y privacidad. El detalle está en el documento de Arquitectura del software.

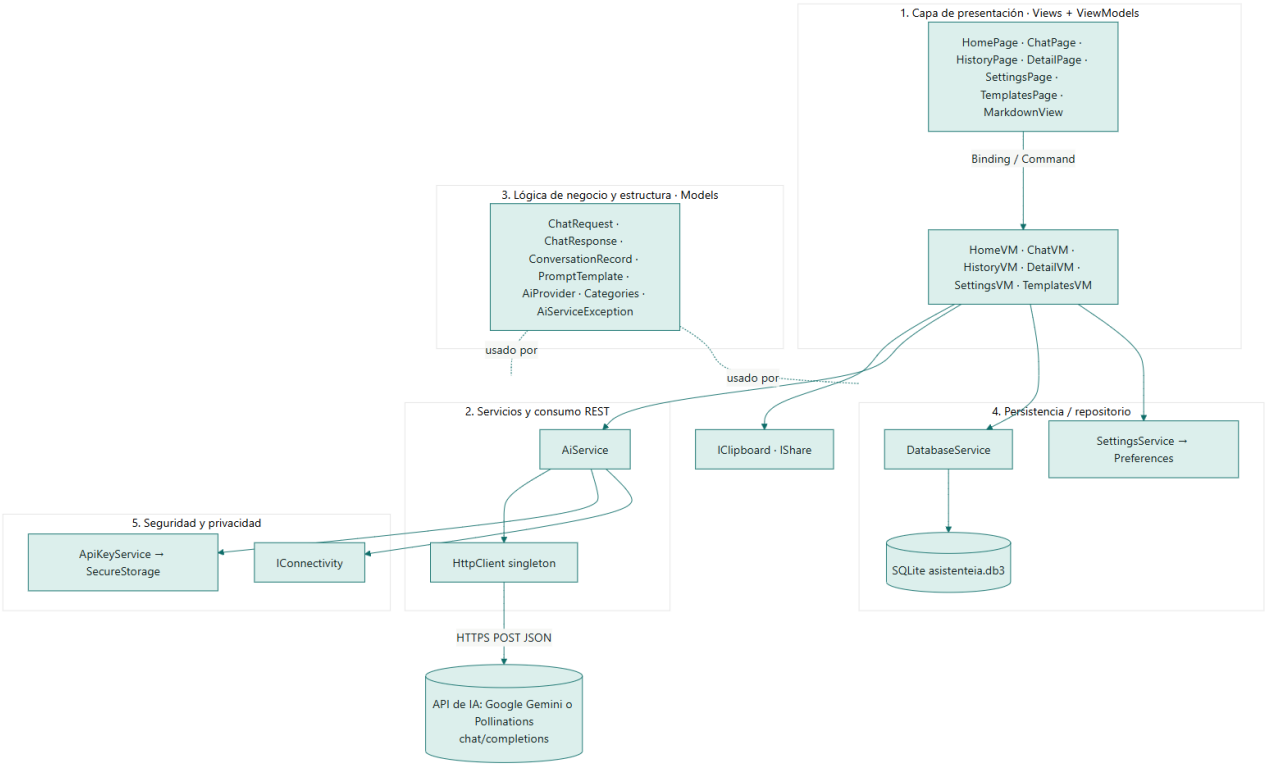


Figura 8. Componentes de la solución y sus dependencias.

AsistenteIA es un asistente académico con inteligencia artificial que consume servicios REST y está organizado en módulos. La solución se divide en cinco componentes. Cada uno corresponde a carpetas y clases concretas del proyecto `AsistenteIA/`.

3. Capa de lógica de negocio y estructura

Contiene las reglas del dominio, los modelos y las decisiones de diseño.

Reglas de negocio

- La consulta debe tener entre 5 y 2000 caracteres (`AiService.ValidatePrompt`). Se valida en el ViewModel antes de enviar y otra vez en el servicio.
- La plantilla elegida se aplica reemplazando `{consulta}` (`PromptTemplate.Apply`). Si el resultado supera 2000 caracteres, se rechaza.
- Solo se guarda una respuesta recibida y no se guarda dos veces (`ChatExchangeViewModel.CanSave`).
- Una respuesta solo se marca como verificada por acción explícita del usuario después de contrastarla con fuentes.
- El tiempo de espera es configurable (de 1 a 120 s) y distinto de la cancelación manual.

Recorrido de una consulta (XAML → ViewModel → REST → JSON → SQLite)

1. XAML. En `ChatPage`, el `Editor` está enlazado a `Prompt` y el botón Consultar a `AskCommand`.

2. ViewModel. `ChatViewModel.AskAsync` valida, aplica la plantilla, agrega un `ChatExchangeViewModel` pendiente a `Exchanges` (burbuja de la pregunta), crea un `CancellationTokenSource` y pone `IsBusy = true`.

3. REST. `AiService.SendChatAsync` envía el POST con Bearer y el JSON.

4. JSON. `ReadFromJsonAsync<ChatResponse>` devuelve `choices[0].message.content`. Si hay error HTTP, se lanza `AiServiceException`.

5. Vista. `exchange.Answer` y `StatusMessage` cambian y el binding refresca la burbuja; `MarkdownView` muestra el formato.

6. SQLite. El botón Guardar de la respuesta ejecuta `SaveCommand` con ese intercambio, que crea un `ConversationRecord` y llama a `DatabaseService.SaveAsync` (`InsertAsync`).

Decisiones técnicas

Decisión	Justificación
MVVM + inyección de dependencias	Separa la interfaz, el estado y el acceso a datos. Todo se registra en `MauiProgram.cs`: servicios como singleton, ViewModels y páginas como transient.
Un solo `HttpClient`	Evita el agotamiento de sockets y centraliza la URL base.
Tiempo de espera propio con CTS enlazado	Distingue el tiempo agotado (P05) de la cancelación manual (P06).
Errores tipados (`AiErrorKind`)	Mensajes claros por cada caso sin exponer detalles técnicos ni la clave.
Proveedor seleccionable, modelo configurable y lista dinámica	Reduce la dependencia del proveedor. Se agregó Google Gemini cuando el equipo obtuvo una clave de Google (F5).
Reintento automático ante 503/429	En las pruebas, Gemini respondió 503 por saturación con frecuencia. Dos reintentos con espera creciente resuelven la mayoría sin intervención (F9).
Markdown interpretado en la vista, no en el prompt	Pedirle al modelo "sin Markdown" hizo que omitiera las tildes (F8). La respuesta se guarda tal como llega y el control `Controls/MarkdownView` la muestra con títulos, negritas, viñetas y bloques de código, una etiqueta por párrafo (F16, F18). `PlainText` quita `#`, `**`, `---` para copiar, compartir y las vistas previas.
Identidad visual propia	Paleta verde azulado con contraste WCAG AA verificado en tema claro y oscuro, fuente Inter, íconos Material Icons Round (fuentes con licencia OFL / Apache 2.0), ícono y splash propios. Los colores de estado usan `AppThemeBinding` + `DataTrigger` sobre `IsError` (F17).
Consulta en formato de conversación	`ChatPage` muestra cada consulta como burbuja del usuario y la respuesta como burbuja de la IA (`ChatExchangeViewModel`). Cada respuesta tiene Copiar, Compartir y Guardar; `IClipboard` e `IShare` se inyectan desde `MauiProgram`.
Animaciones como propiedades adjuntas (`Behaviors/Animations.cs`)	Entrada de burbujas y tarjetas, efecto al presionar botones y tarjetas, pulso de "Pensando la respuesta…", realce del mensaje de estado y sacudida ante errores. Son solo de la vista: no tocan los ViewModels. `MotionPreferences` las omite si el usuario desactivó las animaciones en Windows o Android (accesibilidad).
Plantillas en SQLite	Son editables (RF07) y persisten.
Bindings compilados (`x:DataType`) + XAML `SourceGen`	Errores de enlace detectados al compilar y mejor rendimiento.
Diseño responsive	`FlexLayout Wrap` en filas de controles, `MaximumWidthRequest="1000"` en todas las páginas, lista y formulario lado a lado en pantallas anchas, `Grid` con columna * para textos largos. Verificado en 400, 820 y 1400 px (F3, F11–F15).